

Detailed Methodology Guide

Contrastive Learning + Vision Transformer for PCam Histopathology Image Classification

Under Limited Labeled Data

1. Project at a Glance

Domain	Medical image processing / breast-cancer histopathology
Dataset	PatchCamelyon (PCam)
Baseline	Vision Transformer (ViT) classifier
Proposed method	Self-supervised contrastive pretraining + ViT fine-tuning
Main variable	Labeled training data: 100%, 50%, 25%, 10%
Evaluation	Accuracy, Precision, Recall, F1-score, ROC-AUC, confusion matrix
Research question	Does contrastive learning help ViT maintain classification performance when labeled medical data is limited?

The purpose is not to invent a completely new Transformer. The project is a controlled, reproducible study comparing a ViT baseline with a contrastive-learning-enhanced ViT under different labeled-data conditions.

2. Overall Methodology Flow

PCam Dataset → Preprocessing → ViT Baseline / Contrastive Learning → ViT Encoder → Classification → Compare Results → 100% / 50% / 25% / 10% Labeled Data

Keep the official PCam train/validation/test separation throughout the project. The test set is used only for the final evaluation.

3. Stage 1 — Dataset Preparation

PCam (PatchCamelyon) contains RGB histopathology image patches with binary labels for tumor/metastatic tissue detection. Use the official train, validation, and test files.

1. Download the image and label files for train, validation, and test.
2. Verify that image and label counts match.
3. Display sample images with labels from both classes.
4. Check class distribution and image dimensions.
5. Keep the official test split untouched until the final evaluation.

Milestone

Your first working notebook should load PCam, show sample images, print shapes/class counts, and confirm that the labels are correct.

4. Stage 2 — Image Preprocessing

- Convert images to tensors.

- Resize images to the input size required by the chosen pretrained ViT.
- Normalize pixel values consistently.
- Use deterministic preprocessing for validation and test data.
- Apply random augmentation only to training data.

Do not use random augmentation during validation or testing. This keeps evaluation stable and comparable.

5. Stage 3 — Data Augmentation for Contrastive Learning

For each training image, create two different views using medically reasonable augmentations. These two views form a positive pair.

Original image → Augmentation A → View 1

Original image → Augmentation B → View 2

Start with simple transformations such as crop/resize, horizontal flip, mild rotation, and modest brightness/contrast variation. Avoid aggressive transformations that could change the medical meaning of the tissue structure.

6. Stage 4 — Build the ViT Baseline First

Build and validate a normal ViT classifier before implementing contrastive learning. This gives you a baseline and proves that the data pipeline works.

Image → Patch Embedding → Transformer Encoder → Classification Head → Tumor / Non-Tumor

- Use a standard pretrained ViT from PyTorch/torchvision or timm.
- Replace the classification head with a two-class output.
- Train on the official training set and tune on validation data.
- Record model, image size, batch size, learning rate, epochs, and random seed.

Baseline milestone: stable training/validation curves and a reproducible validation score.

7. Stage 5 — Contrastive Pretraining

Use a SimCLR-style self-supervised contrastive-learning setup. Labels are not needed for this pretraining objective.

Two augmented views → Shared ViT Encoder → Embeddings → Projection Head → Contrastive Loss

- ViT encoder: learns image representations.
- Projection head: a temporary small network used during contrastive training.
- Positive pair: two views of the same original image.
- Negative samples: views from other images in the batch.
- Loss: a standard normalized temperature-scaled contrastive objective.

Only the training images should be used in this pretraining stage. Never use validation or test images for contrastive pretraining.

8. Stage 6 — Fine-Tune the Contrastive ViT

After contrastive pretraining, remove the temporary projection head and use the ViT encoder as the starting point for classification.

Pretrained ViT Encoder → Classification Head → Labeled PCam Images → Fine-Tuned Classifier

First run the full-data setting. Once it works, repeat the fine-tuning experiment using smaller labeled subsets.

9. Stage 7 — Limited-Labeled-Data Experiments

This is the central research experiment. Vary only the amount of labeled training data.

Scenario	Purpose	Rule
100%	Full-data reference	Same subset for both models
50%	Moderate label reduction	Fixed, reproducible subset
25%	Strong label reduction	Fixed, reproducible subset
10%	Low-label scenario	Fixed, reproducible subset

Use the same labeled subset for the baseline and contrastive ViT at each ratio. Choose subsets with a fixed random seed and maintain class representation.

10. Stage 8 — Fair Model Comparison

Same split + Same preprocessing + Same labeled subset → Baseline ViT vs Contrastive ViT

Keep the comparison fair. Differences should come mainly from the contrastive-learning component, not from different data splits or unrelated training settings.

11. Stage 9 — Evaluation

Metric	Meaning	Why use it
Accuracy	Overall correct predictions	General performance
Precision	Correctness of positive predictions	False-positive behavior
Recall / Sensitivity	Positive cases detected	Important for missed tumor cases
F1-score	Balance of precision and recall	Useful main comparison
ROC-AUC	Class separability across thresholds	Threshold-independent summary
Confusion matrix	TP/TN/FP/FN counts	Error analysis

12. Stage 10 — Result Analysis

Prepare one comparison table and plots showing how both models behave as the amount of labeled data decreases.

Labeled data	Model	Accuracy	Precision	Recall	F1 / AUC
100%	Baseline ViT	—	—	—	—
100%	Contrastive ViT	—	—	—	—
50%	Baseline ViT	—	—	—	—
50%	Contrastive ViT	—	—	—	—
25%	Baseline ViT	—	—	—	—
25%	Contrastive ViT	—	—	—	—
10%	Baseline ViT	—	—	—	—
10%	Contrastive ViT	—	—	—	—

The central interpretation is whether contrastive learning provides greater benefit when labeled data is scarce. Do not assume beforehand that the proposed method will win every setting.

13. Optional Small Ablation Study

If time permits, study one extra factor related to DIP: the effect of augmentation choice on contrastive learning.

- Augmentation set A: crop + flip + mild rotation.
- Augmentation set B: crop + flip + rotation + mild intensity/noise changes.
- Compare performance at one or two label ratios.

14. Suggested Software

- Python
- PyTorch
- torchvision or timm
- h5py for PCam HDF5 files
- NumPy
- Pandas
- Matplotlib
- scikit-learn

A CUDA-capable GPU is recommended. If needed, a managed GPU environment such as Google Colab can be used, depending on your course rules and storage limits.

15. Recommended Implementation Order

6. Load PCam and verify images/labels.
7. Visualize the dataset and class distribution.
8. Build preprocessing and data loaders.
9. Implement and train the ViT baseline.
10. Evaluate the baseline on validation data.
11. Implement two-view augmentation.
12. Implement contrastive pretraining with the ViT encoder and projection head.
13. Fine-tune the contrastively pretrained encoder.
14. Run the 100%, 50%, 25%, and 10% experiments.
15. After all choices are finalized, evaluate on the untouched test set.
16. Create tables, plots, confusion matrices, and error examples.
17. Write findings, limitations, and conclusions.

16. Rules for Reliable Research Results

- Never train or tune on the test set.
- Do not use test images during contrastive pretraining.
- Use exactly the same labeled subsets for the two models at each label ratio.
- Keep validation separate from training.
- Fix random seeds where practical.

- Record all hyperparameters and software/model versions.
- Do not repeatedly change the method based on test performance.

17. Expected Outcomes

The project may show that contrastive pretraining helps more strongly at low label ratios, helps only in some settings, or provides little improvement. Any of these can be valid research outcomes if the methodology is controlled and the interpretation is honest.

18. Suggested Work Plan

Phase	Work	Deliverable
1	Literature + dataset	Notes + working data loader
2	Preprocessing + visualization	Clean pipeline + sample plots
3	ViT baseline	Baseline results
4	Contrastive pretraining	Working contrastive model
5	Fine-tuning	Contrastive ViT results
6	Limited-label experiments	10/25/50/100% comparison
7	Evaluation + analysis	Tables, plots, error analysis
8	Report + presentation	Final paper and slides

19. Final Methodology in Simple Form

**PCam Dataset → Preprocessing → ViT Baseline / Contrastive Learning → ViT Encoder → Classification
→ Compare Results → 100% / 50% / 25% / 10% Labeled Data**

The core contribution is a controlled empirical investigation of whether contrastive learning improves Vision Transformer classification when labeled medical data is limited.

20. Key References for Learning the Methods

- Shiri, R., Reddy, V., & Sun, Y. (2024). Supervised Contrastive Vision Transformer for Breast Histopathological Image Classification. <https://arxiv.org/abs/2404.11052>
- PatchCamelyon (PCam) dataset repository. <https://github.com/basveeling/pcam>
- Chen, T. et al. (2020). A Simple Framework for Contrastive Learning of Visual Representations (SimCLR). <https://arxiv.org/abs/2002.05709>
- Dosovitskiy, A. et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. <https://arxiv.org/abs/2010.11929>
- Khosla, P. et al. (2020). Supervised Contrastive Learning. <https://arxiv.org/abs/2004.11362>

21. Algorithm Choices by Stage

The methodology above describes what we do. This section specifies practical algorithms and techniques that can be used at each stage. These are recommendations rather than fixed requirements; the final choice should be kept consistent throughout the experiments.

Stage	Recommended algorithm/technique	Purpose	Recommended choice for our project
-------	---------------------------------	---------	------------------------------------

1. Dataset handling	HDF5 data loading with h5py; stratified sampling for label-ratio experiments	Load PCam image/label arrays and create reproducible subsets.	Use h5py + PyTorch Dataset/DataLoader. Keep official train/validation/test split.
2. Image preprocessing	Resize, tensor conversion, pixel normalization	Make images compatible with the chosen ViT input format.	Resize 96×96 PCam images to the selected ViT input size; use normalization matched to the pretrained ViT.
3. Image augmentation	RandomResizedCrop, HorizontalFlip, small RandomRotation, ColorJitter; optional Gaussian noise	Create realistic variation for training and contrastive views.	Start with crop/resize + flip + mild rotation + mild color/intensity change. Avoid aggressive transforms.
4. Baseline model	Vision Transformer (ViT), preferably pretrained	Provide the non-contrastive baseline.	Use a standard pretrained ViT such as ViT-B/16; replace the head with binary classification.
5. Contrastive learning	SimCLR-style self-supervised contrastive learning + NT-Xent / InfoNCE loss	Learn image representations without using class labels during pretraining.	Use two augmented views of each training image, a shared ViT encoder, and a small projection head.
6. Embedding learning	CLS-token embedding or pooled ViT representation	Convert each image into a compact feature vector.	Use the ViT CLS token as the image representation unless experiments show a better pooling strategy.
7. Fine-tuning	Transfer learning + cross-entropy loss	Adapt the contrastively pretrained ViT to tumor/non-tumor classification.	Remove the contrastive projection head, add a 2-class classifier, and fine-tune on labeled training subsets.
8. Limited-label experiments	Stratified random subsampling	Measure performance under different label availability.	Use 100%, 50%, 25%, and 10% of the training labels; keep the same subsets for both models at each ratio.
9. Optimization	AdamW optimizer + learning-rate scheduler + early stopping	Train models stably and reduce overfitting.	Use AdamW as the default optimizer; consider cosine decay or ReduceLROnPlateau; tune on validation only.
10. Imbalance handling	Class weights or weighted sampling, if imbalance is observed	Prevent the model from favoring the majority class.	Check PCam class balance first. Use weighted loss/sampling only if needed.
11. Evaluation	Accuracy, Precision, Recall/Sensitivity,	Measure both overall performance and	Use F1 and ROC-AUC as key comparison

	F1-score, ROC-AUC, confusion matrix	tumor-detection behavior.	metrics, with recall reported clearly.
12. Explainability/error analysis	Grad-CAM or attention rollout (optional)	Inspect which image regions influence predictions.	Optional for the final extension; use only after the main experiments are stable.
13. Statistical comparison	Multiple random seeds; mean $\pm$ standard deviation; paired comparison across seeds	Check whether differences are stable rather than due to randomness.	Run at least 3 seeds for final experiments if computing resources allow.

22. Recommended Algorithm Stack for Our Final Project

To keep the project manageable for a 4th-year BSc CSE team, the following stack is recommended as the initial implementation:

Component	Recommended initial choice
Data loading	h5py + PyTorch Dataset/DataLoader
Preprocessing	Resize + tensor conversion + ViT-compatible normalization
Augmentation	RandomResizedCrop + HorizontalFlip + mild Rotation + mild ColorJitter
Baseline	Pretrained ViT-B/16
Contrastive learning	SimCLR-style two-view learning
Contrastive loss	NT-Xent / InfoNCE
Fine-tuning loss	Cross-Entropy Loss
Optimizer	AdamW
Scheduler	Cosine learning-rate schedule
Evaluation	Accuracy + Precision + Recall + F1 + ROC-AUC + confusion matrix
Optional explainability	Grad-CAM or attention rollout

23. What We Will Learn at Each Stage

Stage	Main concepts to learn
Dataset preparation	HDF5, PyTorch Dataset/DataLoader, class labels, data leakage
Preprocessing	Image normalization and augmentation
ViT baseline	Patch embeddings, self-attention, Transformer encoder, classification head
Contrastive learning	Positive/negative pairs, embeddings, cosine similarity, InfoNCE/NT-Xent loss
Fine-tuning	Transfer learning, freezing/unfreezing layers, learning rates
Experiments	Controlled variables, ablation, reproducibility, random seeds
Evaluation	Confusion matrix, precision/recall, F1, ROC-AUC, error analysis
Research analysis	Comparing results across label ratios and interpreting limitations

24. Final Methodology Flow with Algorithm Names

This is the concise flow to use in the proposal/presentation:

